Comparing the 208 state proposals

Exact machine identities and the search below 208 states

Research prepared for Vladimir Reshetnikov

29 October 2026

Abstract

No accepted machine below 208 working states was obtained. The five pinned incoming archives contain nine advertised 208-state outputs, but only **two rooted action-labelled machine graphs**. The variation is seven byte strings and six native-labelled transition functions. Thus the newly supplied descriptions do not provide independently stackable reductions. Both graph classes are exactly those in the previously accepted 208-state release, and every literal current table is checked by a complete state conjugacy and exact finite startup replay.

The attempted 207-state decoder reassociation has a concrete failure on a legal canonical primitive frame. It returns after 538 transitions to the wrong source address and decrements the wrong register, while the parent returns correctly after 568 transitions. This refutes that candidate's universal interpreter contract, not its blank-tape halting equivalence. Additional bounded state-merger and repair searches found no accepted smaller machine. Their negative results are not lower bounds.

This report separates exact finite identities, ordinary mathematical proofs, inherited arithmetic calibration, bounded tests and unresolved obligations. The strict requested target is fewer than 208 working states, including blank-tape initialization. The already accepted 208-state result is a baseline, not a solution to that new target.

Research status The strict target remains unmet. The comparison, concrete rejection, and bounded searches in this report are complete; the missing calibration obligations for the investigated theory replacement are stated explicitly.

Pinned scope `VladimirReshetnikov/ProveIt`, commit `c086b5adaa6228009365768946d691dfc49a20bb`, committed 29 October 2026 at 18:14:37 UTC. Later repository changes are not silently included. Exact input hashes and selected bytes accompany the report.

1 The machine model and the success condition

A working state is a state with two defined transitions, one for each binary tape symbol. The machines use one two-way infinite tape, alphabet $\{0, 1\}$, and left or right unit moves. The initial tape is all zero, the initial head is at coordinate 0, and the initial state is 0. A separate halt label is not counted as a working state. Entry into it ends execution after the row's write and move. A native row `q b w D r` means: read b in state q, write w, move in direction D and enter r.

The current machines have 208 working states and 416 defined rows. Their native halt label is 254; a dense serialization names the working states $0, \ldots, 207$ and halt 208. Large sparse labels do not increase the state count. The same convention would give 414 rows for a 207-state candidate. The blank-tape initializer must remain part of the counted machine.

The user permits replacement of Theory `U` provided the replacement remains equiconsistent with PA. This opens a genuine source-level route: it is not necessary to preserve the old literal axiom code.

It remains necessary to prove both consistency directions for the replacement, to connect its actual encoding with the enumerated proof system, and to prove the compiled blank machine implements that proof search. A lower compiler count without these links is not yet a solution.

The earlier accepted endpoint was the PA-formalizable equivalence

$$\neg\,\mathrm{Halt}(U_{208}, \mathrm{blank}) \quad \longleftrightarrow \quad \mathrm{Con}(\mathsf{U}) \quad \longleftrightarrow \quad \mathrm{Con}(\mathrm{PA}).$$

Here that endpoint is *inherited*, not newly derived from a hash, finite simulation or graph-reachability test. No unconditional nonhalting statement, proof-assistant certificate or state-minimality theorem is asserted.

2 An exact comparison of the current proposals

2.1 The nine outputs reduce to two graph classes

In the following table A denotes the Plain/Joint source graph and B the Compatibility source graph. The labels M1–M9 are local evidence identifiers, not mathematical state names. Full paths, native hashes, dense hashes and source hashes are in `evidence/input_index.json`.

ID	Archive label	Output	Class
M1	ConPA	primary	A
M2	Conditional Context	alternative	B
M3	Conditional Context	primary	A
M4	Entry Context	primary	A
M5	Entry Context	Compatibility	B
M6	History	Compatibility	B
M7	History	primary	A
M8	Earlier accepted research	primary	B
M9	Earlier accepted research	Plain variant	A

The class identifiers are SHA-256 hashes of a deterministic breadth-first serialization of the rooted graph, with outgoing read-0 before read-1 edges. Each edge retains its read symbol, write symbol and direction. Start is fixed at canonical label 0 and halt at -1.

Class A

`ab3b7f35756ab763601fae76e24486751d286b699f2ca7d4bedb9b5ea39fe3eb`

Class B

`baaabd3e109e13e134c05facb1db663bda6a491cf2ea2e7f6ef118beaed0fe99`

The explicit row-by-row comparison, rather than a collision-resistance assumption about those identifiers, establishes the equivalences.

Theorem 2.1 (Literal machine equivalence within each class)**.** *For each pair of current machines in the same class there is a bijection f of their working states and halt label that fixes start and halt and satisfies*

$$\delta_2(f(q), b) = (w, D, f(r)) \qquad \textit{whenever} \qquad \delta_1(q, b) = (w, D, r).$$

Consequently, for every same initial complete tape and head coordinate, the two runs have identical complete tapes and head coordinates at each time, with states related by f. They halt at exactly the same time.

Proof. For either rooted graph, enumerate reachable vertices by breadth-first search, processing read-0 before read-1 edges. The complete finite tables show that all 208 working vertices and halt are graph reachable. Match equally numbered vertices in the two traversals. This is a bijection fixing start and halt. For each of the 416 rows, compare read symbol, written symbol, direction and the mapped target; all equations hold. The package rechecks 3,744 equations, one full table per current output against its class representative, including the representatives' identity equations.

Induct on transition count. At time 0 the tapes and heads agree and states are related. At a nonhalting time, both heads read the same symbol. The checked row equation makes both writes and moves equal and relates the successor states. Halt labels correspond, so neither run can halt without the other. This is a finite-step induction available in PA; it does not assume any consistency or claim that halt is reached from the blank tape. □

The dense tables are checked separately: ascending native working-state order is renamed to $0, \ldots, 207$, and 254 to 208. All 416 dense rows match that renaming for every advertised output. A fresh 48,140-step startup replay also compares the complete finite-support tape, head and corresponding state for each class. This is a useful independent finite regression; it is not a replacement for the all-time conjugacy theorem.

2.2 Why the apparent reductions cannot be stacked

ConPA and History retain native 172 and delete 211. Conditional Context, Entry Context and the accepted research tables retain 211 and delete 172. These are different names for the same contextual fold, not two distinct state savings. In the retained-211 form, the old 172/0 transition is simulated by seven transitions, the old 211/0 by three, and either read-1 transition by one. Renaming the surviving merged state does not leave a second copy available to delete.

Conditional Context and Entry Context have byte-identical native and dense tables for their corresponding variants. The Compatibility native table in those archives and the accepted research primary has the same normalized transition function but a different serialization. Differences between archive size, report structure, state labels and whitespace are not changes to the blank computation.

2.3 What the reports add beyond their shared graphs

The supplied documents emphasize different explanations and checking styles. ConPA and History describe predecessor histories; Conditional Context states an explicit five-state Boolean invariant and enumerated finite obligations; Entry Context uses a short complete incoming-edge census; the accepted research report gives four partial-tape identities and commuting ordinary rows. Those are potentially useful proof presentations of the common fold. They do not alter the measured graph classes.

This comparison reads supplied material as data. It does not execute supplied scripts, adopt their instructions, or count their advertised regression totals as newly rerun evidence. The independent checker included here is newly authored. The nine literal tables, rather than the supplied programs that produced them, are the objects rechecked.

3 A concrete failure of the attempted 207 state repair

3.1 The edit and the misleading local intuition

The provisional attempt rotates decoder roots 155 and 52, makes the rows of 85 and 154 coincide, merges 154 into 85, and redirects 118/0 to 186. The candidate has 207 working states and can pass the exact finite startup. Its source word and ordinal control-flow graph can also remain unchanged. Neither fact establishes preservation of shared arithmetic entrances.

At 118/0, the proposed route through 186 and 147 mimics an old route through 155 and 85 only under an extra assumption on the next cells. The required assumption is false even on a canonical frame. At source ordinal 70, the address is `0011001001100`, the instruction is CD_5, and an actual local segment is

$$134_7 \to 76_8 \to 152_7 \to 159_6 \to 118_7 \to 155_8 \to 85_9 \to 12_{10}.$$

At state 118_7, cells 8 and 9 contain 0 and 1. The replacement reads that 1 at 147 and enters 166 instead of the old entry 12. An all-zero-register test can hide the resulting error; a nonzero unrelated register exposes it.

3.2 The complete canonical counterexample

Use initial state 84, head 0, gap 0, spare list [0], source ordinal 70 and logical register vector

$$(R_0, \ldots, R_8) = (0, 0, 0, 1, 0, 0, 0, 0, 0).$$

The complete tape is zero except at the following coordinates:

$$\{-1, 2, 3, 6, 9, 10, 56, 58, 60, 62, 64, 66, 67, 69, 71, 73, 75, 77\}.$$

This is the ordinary canonical encoding: the address begins at coordinate 0, the header is at 56, and logical registers use the physical permutation $(6, 2, 0, 4, 3, 7, 8, 5, 1)$ with unary blocks, followed by the spare zero block. No unconstrained or symbolic cells remain in this example.

Canonical first return	Parent	Rejected candidate
Transition count	568	538
State and head	84 at 0	84 at 0
Stored PC without trailing zeros	`001100100111`	`0011001001101`
Resolved source leaf	`0011001001110`	`00110010011010`
Resolved ordinal and instruction	74, CD_6	71, CI_1
Header coordinate	56	57
Logical R_3	1	0
All other logical registers	0	0
Spare list	[0]	[0]

Because $R_5 = 0$, the parent correctly takes the zero branch without decrement. The candidate instead decrements logical R_3, changes the header position and chooses a different source address. The package reconstructs the exact input tape, replays every row up to both first canonical returns, validates each source address and every decoded register block, verifies that no earlier full canonical return occurred, and compares every final one-cell. A trace digest pins each complete replay without including redundant large JSON trace files.

Theorem 3.1 (Rejected universal primitive claim)**.** *The displayed 207-state candidate does not implement the parent's canonical* CD_5 *contract on all legal frames at ordinal 70.*

Proof. The specified input satisfies the canonical-frame definition and $R_5 = 0$. The literal deterministic replay yields the two different complete canonical outputs shown above. A universal contract requires the zero branch to leave all logical registers unchanged and select the specified source successor. The candidate violates both requirements on this one input. □

Scope of the rejection. A legal canonical input need not occur on the unique blank computation. This counterexample therefore does not prove that the candidate and parent halt differently from blank, or that the candidate cannot reach the expected frame at some later time. It does rule out the claimed universal primitive theorem used to justify the construction. A new blank-run-specific invariant might avoid the bad frame, but none has been proved here. Bounded predecessor enumeration cannot supply that missing proof merely by failing to find a bad history.

4 What the bounded searches established

4.1 Expanded table searches

The following searches were run on both verified 208-state classes or on the explicitly rejected 207-state candidate. An action trial is an inner search operation, not a distinct machine certified correct. Counts from overlapping or preliminary searches must not be added as if they described disjoint candidate spaces.

Search	First trials	Further trials	Scope
One edited row B	117,576	—	Generalized unchanged-branch endpoint
One edited row A	116,748	—	Same method
Coupled rows B	35,307,576	73,428,696	Both merged branches editable
Coupled rows A	35,307,576	73,026,288	Same method
Halting pair each	342,784	840,320	Pairs involving native 68 only
Repair 118 plus helper	341,964	1,554,984	Rejected candidate only
Repair 118 plus both 85 rows	685,584	54,648	Rejected candidate only

All entries returned no hits. The coupled-row searches use depth-8 predecessor contexts, at most 40 transitions, head radius 8 and a 1,000-node symbolic budget per context. The repair searches use at most 60 transitions, radius 8 and 2,000 nodes. Reaching a limit is rejection by that bounded search, not a mathematical obstruction. The one-row search tests all assignments of newly read cells but has its own branch-independence precondition; it is not an arbitrary-program search. The full method and raw counts are in `search/table/README.txt` and the corresponding logs.

The halting-pair search deliberately permits different endpoint head and tape when both paths halt. It still considers only pairs involving native 68, the sole parent state with an immediate halt row. The three-row repair search separates the old and rotated entrances to 85 by their preceding bit. A positive result from either repair search would still require final-table invariants, source transport and universal runtime verification; no positive result occurred.

4.2 Two sided macro aliases and source reassociations

A second independent extension allows both old and new executions to take multiple transitions before matching. For each class it considers all 42,849 ordered pairs (q, r) with $q \neq 0$ and $r \neq q$, deletes q, redirects targets to r and looks for equal complete tapes, heads and states at positive prefix lengths. Temporary writes may be restored before a match. A five-state positive control succeeds at a two-old-step match that a one-old-step matcher misses.

The actual 208-state runs have no accepted aliases. The tested bounds are 8 or 12 backward edges, 16, 32 or 64 transitions per side, head radius 30 and 3,000 symbolic nodes per context. These parameter combinations overlap; the 342,792 parameterized trials are not that many distinct state identifications. The program, raw context inputs, positive control and run receipts are in `search/macro/`. Limits remain rejection limits, not completeness claims.

On the source side, a broader reassociation criterion checks the whole resolved ordinal CFG rather than requiring three locally closed factors. Of 70 initial structural rotations, 57 preserve the CFG, versus 50 that satisfy the older closed-factor condition. Only two of the seven extra rotations preserve the exact startup address in the initial layout. A random-walk search seeded by those two ran 6,640 iterations in about 302 seconds without a smaller accepted candidate. An iteration is not a distinct source layout. Eight regroupings of three disjoint increment-commutation pairs attain no fewer than the baseline 197 source DAG nodes. The selected source receipts record these bounded outcomes; none proves global optimality.

4.3 Sound source abstractions do not prove concrete reachability

A ternary abstract source execution uses values 0, 1 and ≥ 2 for each of the nine registers. Starting at ordinal 136 with abstraction $(1, 0, 2, 0, 0, 1, 0, 0, 1)$, its closed reachable set has 4,513,459 abstract states. For the post-startup ordinals 136 onward, the absent ordinals are precisely 3848, 3849 and 3892; reached decrements 3847 and 3891 have only zero inputs. All individual per-ordinal register masks are supplied. Excluding those three ordinals and the two impossible positive branches leaves 5,079 guarded primitive contracts. For each class they still cover all 416 transition rows. The stronger per-ordinal register-mask restrictions likewise leave no unused row; 11,305 register constraints were applied in each class. These coverage results concern the sound abstract restriction, not concrete reachability.

Lemma 4.1 (Soundness of the ternary overapproximation)**.** *Every concrete source configuration reached from the specified startup maps to an abstract configuration in the computed closure, provided the concrete source control-flow graph is the checked graph used by the abstraction.*

Proof. Map a register value n to $\alpha(n) = 0$ for $n = 0$, to 1 for $n = 1$, and to 2 for $n \geq 2$. Increment maps 0 to 1, 1 to 2 and 2 to 2. A zero-test follows the zero edge exactly when the abstract value is 0. Positive decrement maps 1 to 0 and permits either 1 or 2 from abstract value 2: these cover concrete predecessors 2 and at least 3 respectively. Control-only commands preserve the abstract registers. Thus every concrete step is represented by an abstract edge. The initial abstraction is included; induction on concrete step count proves containment in the closed set. □

The converse is false in general because saturation and independent branching lose information. In particular an abstract visit to ordinal 70 with a matching mask is not proof that the displayed concrete counterexample lies on the blank computation. The abstraction can safely exclude cases absent from its closure, but cannot establish every present case is realizable.

5 An investigated theory replacement and its remaining obligations

5.1 Weak and exact filtered adjunction

Work first in ordinary classical first-order logic with equality and binary relation E. For each formula $\varphi(x, \bar{p})$, with fresh output variable b, put $A(x, a, y) \equiv E(x, a) \vee x = y$. Weak filtered adjunction is the schema

$$\forall a\, \forall y\, \exists b \big[\forall x\, (E(x, b) \to \varphi(x, \bar{p})) \\ \wedge\, \forall x\, (A(x, a, y) \to (\varphi(x, \bar{p}) \to E(x, b)))\big],$$

with universal closure over any further parameters. It permits extra members of b that satisfy φ but are not in $a \cup \{y\}$. Exact filtered adjunction instead requires

$$\forall a\, \forall y\, \exists b\, \forall x\, \big(E(x, b) \leftrightarrow (A(x, a, y) \wedge \varphi(x, \bar{p}))\big).$$

Lemma 5.1 (Effective equivalence of the two schemas)**.** *With the same underlying first-order logic and any common additional axioms, the complete weak and exact filtered-adjunction schemas prove one another.*

Proof. An exact witness satisfies each implication required of a weak witness, so each weak instance follows from the corresponding exact instance. Conversely, given φ, instantiate the weak schema with $\psi(x) = A(x, a, y) \wedge \varphi(x, \bar{p})$. Its upper clause gives $E(x, b) \to A(x, a, y) \wedge \varphi(x, \bar{p})$. Its lower clause gives $A(x, a, y) \to ((A(x, a, y) \wedge \varphi(x, \bar{p})) \to E(x, b))$, which entails the opposite implication. These two implications give the exact biconditional. Freshen b first to avoid capture. Each step is a finite syntactic derivation, so the transformation is effective on instances and finite proofs. □

This lemma is a theory-to-theory equivalence, not a proof of either theory's PA-equiconsistency. A primary lead is Harvey Friedman, *Issues in the Foundations of Mathematics*, Gödel Lecture, 2 June 2002, section C2, page 10. It states mutual interpretability with PA for a four-atomic-formula adjunction schema. The displayed bound and free variables appear inconsistent, and the lecture gives no lower interpretation proof there. This research does not treat that assertion or a silently corrected formula as an audited finite-refutation translation.

5.2 A measured emitter using the fixed object label 5

The source-cost experiment uses source variable v_0, tested variable v_1, adjoined-object variable v_5 and output binder v_c. In the inherited raw formula encoding, I constructs implication, R membership, Q universal binding and Eq equality; N abbreviates implication to the fixed false code. The proposed lower guard encodes $E(v_1, v_0) \vee v_1 = v_5$. Its postfix word is

```
c 1 1 0 R N 1 5 Eq I c 1 c R I I Q
1 1 c R c I Q N I Q
```

There are 27 tokens. The experiment uses a retained K215/U222 reference pipeline with fixed source/test/output roles, preserving that reference shell and its membership-vacuity bundle. It is not a pointwise rewrite of the later accepted 208-state source, which uses a different role assignment and additional source transport. It is a source experiment, not a new literal machine.

The choice 5 is semantically motivated, not just a cheaper constant. A naive choice of fixed object label 2 fails at code $c = 2$: the decoded matrix is a tautology while the output and adjoined-object variables

coincide, forcing a self-member contrary to irreflexivity. For label 5, the special decoded matrix is $E(v_1, v_0)$; when $c = 5$, choosing the output equal to the source makes both clauses valid. The other small collisions are $c = 0$ and $c = 1$. In the intended ordinary finite-set model, the former can use a singleton container and the latter has a false matrix under irreflexivity. These are local model checks, not the missing proof-calculus translation.

Interval dynamic programming on the inherited closed macros gives 723 emitter primitive occurrences, 3,877 whole-source primitive occurrences and **207 distinct source DAG nodes**, compared with 204 DAG nodes for the original emitter under the same closed-macro compiler and interval-DP procedure. The separately optimized accepted source has 197 DAG nodes. The DAG count is the number of distinct source subexpressions, not the number of working Turing-machine states. Fewer tokens or occurrences did not give a smaller shared source representation in this experiment. A same-theory experiment also factors the two universal cover clauses under one binder, with the outer negation preserved. Under the same compiler, the original has 28 tokens, 737 emitter primitives, 3,891 total primitives and 204 DAG nodes. Factoring keeps 28 tokens but gives 782 emitter primitives, 3,936 total primitives, and 205 or 206 DAG nodes depending on conjunct order. This controlled family therefore did not improve the original. The accepted 197-node, 3,909-primitive source uses the later ST215 exterior and a different role-based emitter: its 3,909 occurrences are $4,732 - 1,597 + 774$, whereas the reference original has $4,751 - 1,597 + 737 = 3,891$. The difference 18 is therefore $37 - 19$, not an unexplained count discrepancy. The factoring equivalence is within the measured reference role assignment; no pointwise equivalence across role assignments or cost for the current literal emitter is inferred.

A syntactic cost sweep with object labels 2 through 16 attains DAG counts from 207 to 211. It does not establish all those alternative theories are sound; in particular label 2 has the collision just described.

5.3 Why this is not yet an accepted replacement

Three obligations remain distinct. First, the exact intended theory needs both audited finite-refutation translations linking it with PA. Second, the new equality guard needs a bridge through the actual numerical calculus. The inherited ordinary-logic interpretation uses equality as membership indiscernibility; equality-free old axioms did not present this additional guard obligation. Ordinary-equality model checks do not discharge it. Third, a proposed source must be physically allocated and connected to a literal smaller TM with all-time interpreter and exact blank-startup proofs. No such accepted machine is supplied by this branch.

6 Reproduction and evidence boundaries

The archive contains selected literal machine tables, two deduplicated current source maps, the rejected candidate and its witness, compact receipts, the new comparison checker, search source text and this full LaTeX report. It does not contain compiled search executables or duplicate historical PDFs. Input archives are identified by hashes rather than copied wholesale.

The fast independent replay is

```
python3 verify.py -negative-controls
python3 -O verify.py -negative-controls
```

The checker uses explicit exceptions, not Python assertions. It recomputes state counts, dense renamings, all graph conjugacies, exact startups and the complete rejected-candidate witness. It also runs negative

controls that alter a transition write and the input frame. A passing replay means precisely that these comparison and rejection obligations passed; it does not turn the rejected 207-state table into a valid machine.

The C++ search sources are reproducible independently with a C++17 compiler. For example, from the package root:

```
c++ -O3 -std=c++17 search/table/coupled_all.cpp -o /tmp/u208-search
/tmp/u208-search search/table/4564c487.dat
```

The selected raw context inputs make this run independent of the original workspace layout. Historical helper scripts retained for provenance may use workspace-relative paths; they are not the standalone fast verifier.

The release verification receipt records manifest checking, normal and optimized replays from a clean extraction, a rebuild of this PDF, equality of extracted text and rendered page pixels, and page-by-page visual inspection. These release checks establish artifact integrity and the stated finite obligations, not a proof of arithmetic consistency.

7 What remains necessary for a smaller result

A successful continuation must provide a literal machine with at most 207 working states, a proved correct blank initializer, and a complete path to the chosen theory's inconsistency search. If Theory U changes, its exact new axioms and both consistency directions must be explicit. If source addresses change, all shared arithmetic and decoder entrances must be transported, not only the visible source control-flow graph. If a local reduction uses a guard, that guard must hold at every relevant boundary of the run to which the simulation theorem applies.

The comparison removes one misleading avenue: the current 208-state reports do not contain independent folds that can simply be combined. The concrete counterexample removes another: startup and ordinal-CFG preservation alone do not validate the proposed decoder rotation. These are useful constraints on the next construction, but neither is a proof that 208 is optimal. **The requested fewer-than-208 result is not achieved in this report.**

A Input archive identities

These are the exact downloaded ZIP bytes at the pinned commit. The full archives are not repeated inside this compact package.

`U208_ConPA_2026-10-29.zip`
12680643 bytes. SHA-256

`0662a968902f231ac2483bbd424f80b8f86e6fe1e04bbe97b6f25c634aa532f9`

`U208_ConPA_Conditional_Context_2026-10-29.zip`
12261070 bytes. SHA-256

`6f23aaca025984a9a3561c7cf352cbde069096713269d8ace9b8c0e305f3b251`

`U208_ConPA_Entry_Context_2026-10-29.zip`
14793535 bytes. SHA-256

`00b0a699606a654bcc8b2079932ec3c6c7303b0adc019e548db2b12d62eb4868`

`U208_ConPA_History_2026-10-29.zip`
21608661 bytes. SHA-256

`2479f562ba30b68bcabcc9ab481cdcb20da2a74476bff132a99382a6669a88aa`

`u208_research_2026-10-29.zip`
3962517 bytes. SHA-256

`33666892d7e729084edb5aa6e2b5e07d0117adbb63b0f46baff5bbbdf1da3d09`